FLAG

FLAG · ESPECIALIZADO FRONT-END WEB DEVELOPMENT

Introdução à Programação

Sessão 1 · Arranque e fundamentos de programação

2.ª feira, 28 set 2026 · 9:30–12:30 · Live Training (MS Teams)

João Gonçalves · joaogoncalves.net

Boas práticas Live Training



Escolher um sítio calmo, sem distrações



Ser pontual



Desligar microfone. Ligar quando for para intervir



Manter a câmara ligada



Levantar a mão quando pretender participar



No intervalo, desligar a câmara e o microfone



“I’ve always believed the web is for everyone. That’s why I and others fight fiercely to protect it.”

Tim Berners-Lee, criador da World Wide Web

ANTES DE COMEÇAR

Quem vos acompanha



João Gonçalves

Interactive Developer · Trainer · Consultant

- Licenciatura em Matemática Aplicada · Mestrado em Comunicação Multimédia
- Formação e consultoria em front-end há décadas: HTML, CSS, JavaScript, Angular e React

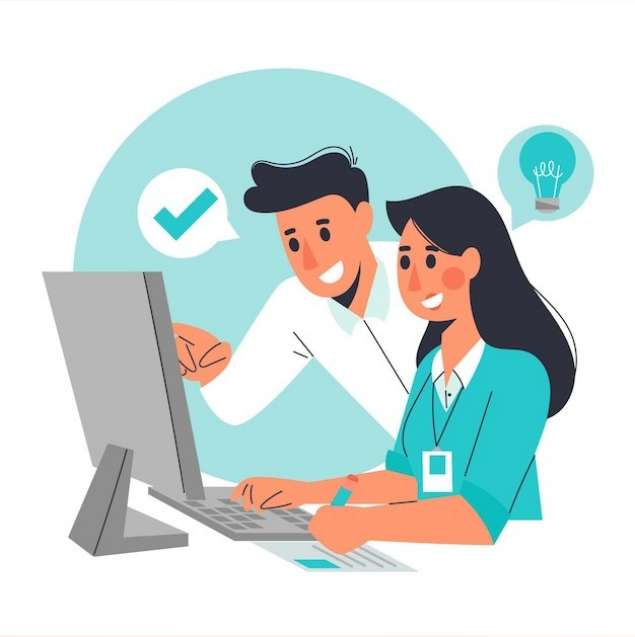
Contactos

edu@joaogoncalves.net
WhatsApp: +351 910 346 437
joaogoncalves.net · @joaopapin

“Making the complicated simple.”

RONDA RÁPIDA

E vocês?



1

Nome e percurso

De onde vêm? Área de formação ou profissão atual.

2

Já programaram?

Nunca, um pouco (Excel, scripts, HTML), ou com regularidade?

3

Porquê front-end?

O que vos trouxe a esta especialização?

4

Uma expectativa

O que querem conseguir fazer em março, no fim do curso?

COMO VAMOS TRABALHAR

Ritmo, ferramentas e método

1

2.ª · 4.ª · 6.ª · Teams

Live Training, 9:30–12:30, com pausa de 10–15 min a meio. No módulo de Projeto: 9:30–11:00.

2

Canal Teams da turma

Chat para links, dúvidas entre sessões e snippets. Perguntar cedo é sempre melhor.

3

Repositório GitHub

O código de cada sessão fica publicado. A partir do módulo Git: git pull no início de cada aula.

4

Conceito

Explicação curta com exemplos reais e atuais.

5

Demo ao vivo

Partilha de ecrã: construímos juntos, passo a passo — erros incluídos.

6

Exercício

Prática guiada na sessão; desafios opcionais para casa.

PARTILHA

Onde vive o material

1

MS Teams

As sessões e o chat da formação: avisos, links rápidos e dúvidas durante e entre sessões.

2

Moodle FLAG

Os materiais oficiais de cada sessão ficam no Moodle, no fim do dia. Fórum da turma.

3

GitHub

Slides, exemplos e código feitos em aula.
github.com/JoaoGoncalves/flag-frontend-2026

4

WhatsApp / telefone

Para falar comigo por outro motivo:
+351 910 346 437

O PERCURSO

8 módulos, 28 set 2026 → 29 mar 2027

#	Módulo	Sessões	Datas	O que sai daqui
01	Introdução à Programação e Desenvolvimento Web	6	28 set – 12 out	Pensar em algoritmos; perceber a Web
02	HTML e CSS	12	14 out – 9 nov	Micro-site responsivo e acessível
03	Workshop: Versões de Código com GitHub	3	11 – 16 nov	Portfólio versionado e publicado
04	JavaScript para Frontend	20	18 nov – 22 jan	Apps interativas com DOM e APIs
05	React Foundations	9	25 jan – 15 fev	SPA com componentes e hooks
06	React Avançado	4	17 – 24 fev	Gestão de estado com Redux Toolkit
07	Pitstop	1	26 fev	Revisão e arranque do projeto
08	Projeto: Front-End	12	1 – 29 mar	Aplicação React para o portfólio

MÓDULO 01

As 6 sessões deste módulo

S1 — **28 set**

Fundamentos

O que é programar · linguagens · ordem · algoritmos

S2 — **30 set**

Portugol

Os primeiros algoritmos a correr: sequência, decisão e repetição

S3 — **2 out**

Algoritmos I

Variáveis, tipos, operadores, pseudocódigo e fluxogramas

S4 — **7 out**

Algoritmos II

Ciclos, vetores, teste de mesa e estudos de caso

S5 — **9 out**

Arquitetura da Web

Cliente/servidor, IP, DNS, HTTP, cloud e alojamento

S6 — **12 out**

Front-end e Back-end

Responsabilidades, linguagens, APIs e bases de dados

5 out é feriado (Implantação da República): sem sessão.

FLAG

FICA A PERGUNTA

Internet e Web são a mesma coisa?



Pensem nisto até à sessão 5.

A 9 de outubro vamos ver como a Internet funciona por dentro: redes, IP, DNS, protocolos e servidores. E onde é que a Web entra nisso tudo.

PARTE 1

O que é programar?

Programação vs linguagem · da ideia ao software

UMA DEFINIÇÃO DE TRABALHO

Programar é…

…descrever a solução de um problema com instruções tão precisas que uma máquina as consegue executar sem interpretar.

O computador é rápido e incansável, mas não adivinha intenções

Faz exatamente o que lhe dizemos, não o que queríamos dizer.

FLAG

ATIVIDADE · 10 MIN

O robô que faz sandes

Cada um escreve as instruções para fazer uma sandes de manteiga. O formador executa-as à letra, em frente à câmara.

1. 5 minutos: cada um escreve os seus passos (máx. 10), sem ver os dos outros, e cola-os no chat da reunião.
2. O formador lê cada algoritmo e "executa-o" literalmente na câmara, sem preencher lacunas.
3. Identificamos juntos onde falhou: passos em falta, ambiguidade, ordem errada.
4. Juntos, escrevemos uma versão final que o robô consegue executar.

O objetivo não é a sandes: é descobrir quanta informação "óbvia" temos de tornar explícita.

O que vão descobrir

"Pegar no pão" — qual pão? Com que mão? Está dentro do saco?

Um computador precisa de passos completos, na ordem certa e sem ambiguidade. É exatamente isto que um algoritmo resolve.

Uma solução, várias línguas

Programação é o raciocínio: decompor o problema e ordenar os passos. A linguagem é a ferramenta com que escrevemos esse raciocínio, cada uma com a sua sintaxe.

PSEUDOCÓDIGO (PORTUGUÊS)

```
ler nota
se nota >= 10 entao
  escrever "Aprovado"
senao
  escrever "Reprovado"
fimse
```

JAVASCRIPT

```
const nota =
Number(prompt("Nota?"));
if (nota >= 10) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
```

PYTHON

```
nota = float(input("Nota? "))
if nota >= 10:
    print("Aprovado")
else:
    print("Reprovado")
```

Mesma lógica: ler, decidir, escrever. Muda a sintaxe, não o algoritmo.

DA IDEIA AO SOFTWARE

Os termos que vamos usar

1

Lógica

Encadear pensamentos para atingir um objetivo.

2

Sequência lógica

Os passos, por ordem, até à solução.

3

Instrução

Uma ação elementar que o computador executa.

4

Algoritmo

Sequência finita de instruções que resolve um problema.

5

Programa

Um algoritmo escrito numa linguagem de programação.

6

Software

Conjunto de programas (+ dados, + interface).

Como é que o computador percebe o nosso código?

	Compiladas	Interpretadas / JIT
Como funcionam	Todo o código é traduzido para linguagem-máquina antes de correr	O código é traduzido enquanto corre (hoje quase sempre com compilação JIT)
Exemplos	C, C++, Rust, Go	JavaScript, Python, PHP, Ruby
Vantagem	Execução muito rápida, erros detetados cedo	Ciclo “escrever → testar” imediato, multiplataforma
Onde vamos estar	—	JavaScript: no browser (front-end) e no Node.js (ferramentas e back-end)

Alto nível = mais perto da linguagem humana. Baixo nível = mais perto do hardware.

PARTE 2

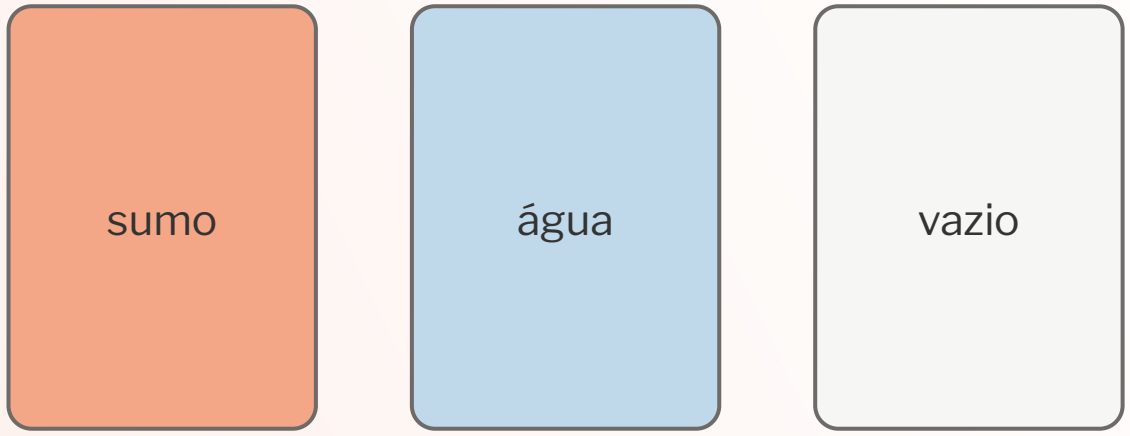
Conceitos de “ordem”

Sequência · decisão · repetição

A ORDEM IMPORTA

Trocar o conteúdo de dois copos

O copo A tem sumo e o copo B tem água. Como trocamos os conteúdos?



ALGORITMO

```
1. deitar A em AUX    // AUX = sumo
2. deitar B em A      // A   = água
3. deitar AUX em B    // B   = sumo
```

Troquem a ordem dos passos 1 e 2 e o sumo perde-se. Em programação, a ordem das instruções é parte da solução.

FLAG

TRÊS ESTRUTURAS CHEGAM

Todo o algoritmo é feito de três blocos

1

Sequência

Instruções executadas uma a seguir à outra, pela ordem em que aparecem.

“Ligar o carro, engatar, arrancar.”

2

Decisão

Escolher um caminho conforme uma condição (verdadeiro/falso).

“Se estiver em zona pública, ligar os quatro piscas.”

3

Repetição

Repetir instruções enquanto (ou até que) uma condição se verifique.

“Enquanto houver parafusos, desapertar.”

Böhm & Jacopini (1966): sequência, decisão e repetição são suficientes para expressar qualquer algoritmo.

PENSAMENTO COMPUTACIONAL

Os 4 pilares de quem programa

Não é “pensar como um computador”: é pensar de forma a que um computador possa ajudar.

1

Decomposição

Partir um problema grande em problemas pequenos e resolúveis.

2

Padrões

Reconhecer semelhanças com problemas que já resolvemos.

3

Abstração

Ignorar o detalhe irrelevante e ficar com o essencial.

4

Algoritmo

Escrever a solução como uma sequência de passos.

Exemplo: organizar uma festa

decompor (comida, convites, local) → padrões (como foi a última) → abstrair (n.º de pessoas, orçamento) → plano.

PARTE 3

Algoritmos

Características · entrada, processamento e saída · como escrever um

FLAG

ALGORITMO

Uma receita que não deixa margem para dúvidas

Um algoritmo é uma sequência finita de passos claros e objetivos que resolve um problema.

Receita de cozinha: ingredientes (entrada), passos (processamento), prato (saída).

Manual de montagem de um móvel: ordem estrita, peças identificadas.

GPS: calcula o caminho mais curto entre dois pontos (algoritmo de Dijkstra e derivados).

Feed de uma rede social: ordena publicações segundo regras (o famoso "algoritmo").

Construir algoritmos é programar.

Ao contrário de uma linguagem de programação, não há um formalismo rígido para escrever um algoritmo: pode ser texto, pseudocódigo ou um fluxograma. O que conta é ser preciso.

CARACTERÍSTICAS

As 5 características de um algoritmo

1

Finitude

Tem de terminar. Nenhum passo pode ficar a repetir-se para sempre.

2

Definitude

Cada passo é definido sem ambiguidade. Lembram-se da sandes?

3

Entrada

Identifica os dados de que precisa (zero ou mais).

4

Saída

Produz pelo menos um resultado: é para isso que existe.

5

Eficácia

Cada passo é simples o suficiente para ser executado e verificado.

Na atividade da sandes: ambiguidade = falha de definitude; passo impossível = falha de eficácia.

FLAG

FASES

Entrada → Processamento → Saída

ENTRADA

Os dados de que precisamos

```
nota1, nota2, nota3
```

→

PROCESSAMENTO

O que fazemos com eles

```
media ← (n1 + n2 + n3) / 3
```

→

SAÍDA

O resultado apresentado

```
"A média é 14,3"
```

Na Web

entrada = clique ou formulário · processamento = JavaScript (e/ou servidor) · saída = o que muda no ecrã.

MÉTODO

Como escrever um algoritmo

1

Definir o objetivo

O que queremos alcançar? Quais são as entradas e as saídas?

2

Conceber o algoritmo

Dividir o objetivo em pequenas instruções, pela ordem certa.

3

Testar no papel

Seguir os passos com valores concretos. Funciona em todos os casos?

4

Codificar

Traduzir cada instrução para uma linguagem de programação.

Os passos 1–3 fazem-se sem computador. É aqui que se ganha (ou perde) mais tempo num projeto real.

EXERCÍCIO · 25 MIN

Mudar um pneu

Primeiro sozinhos (15 min), depois juntos (10 min): o algoritmo para mudar um pneu furado, com entradas, saídas e sequência numerada.

1. Listem os dados de entrada (o que é preciso ter/saber antes de começar).
2. Listem as saídas (como fica tudo no fim).
3. Escrevam a sequência algorítmica numerada, do “parar o carro” ao “seguir viagem”.
4. Assinalem com [SE] os passos que dependem de uma condição e com [REPETE] os que se repetem.

Depois, juntos

Comparamos as três versões e construímos uma só, a melhor. Cada um entrega o seu ficheiro no canal Teams (ex.: mudar-pneu-ana.md).

MUDAR-PNEU.MD (INÍCIO)

```
## Entradas
- Pneu novo, macaco, chave
- Triângulo, colete

## Saídas
- Pneu trocado
- Ferramentas arrumadas

## Sequência
1. Parar o carro em local seguro
2. Desligar o motor
3. ...
```

FLAG

DISCUSSÃO

O pneu já tem decisão e repetição

EXCERTO DE UMA RESOLUÇÃO DE TURMA ANTERIOR

```
1.  Parar o carro numa localização segura
2.  Desligar o carro em segurança
3.  Se estiver numa zona pública,
      ligar a sinalização de emergência
...
15. Dirigir-me ao pneu furado
16. Enquanto houver parafusos,
      desapertar os parafusos
...
27. Enquanto houver parafusos,
      apertar os parafusos
28. Baixar o macaco
```

Sequência: quase todos os passos. A ordem é crítica (desapertar antes de levantar o carro!).

Decisão: passo 3, “Se estiver numa zona pública…”.

Repetição: passos 16 e 27, “Enquanto houver parafusos…”.

Abstração: “apertar os parafusos” esconde muitos sub-passos. Tudo bem, desde que o executor os saiba fazer.

ATÉ QUARTA-FEIRA

Preparar a sessão 2

1

Instalar o Portugol

Java + Portugol, seguindo o guia do próximo slide. Testar o primeiro programa antes da aula.

2

Aquecer com Lightbot

Resolver os primeiros níveis de um jogo de lógica: Lightbot ou os puzzles da Hour of Code (code.org).

3

Um algoritmo do vosso dia

Escolham uma tarefa diária e descrevam entradas, processamento e saída. Partilhar no canal Teams.

Próxima sessão · 4.ª, 30 set

Pensamento computacional com Portugol: os primeiros algoritmos a correr num computador.

INSTALAR ATÉ QUARTA

Portugol: download e instalação

1

Java (LTS)

Eclipse Temurin, versão 21 ou mais recente. Mac com chip Apple: aarch64 · Mac Intel e Windows: x64.

adoptium.net

2

Portugol

Descarregar o portugol23.zip e extrair a pasta inteira (o Portugol.jar precisa da pasta lib ao lado).

GitHub › recursos · Moodle

3

Abrir e testar

Duplo clique em Portugol.jar, ou no terminal: java -jar Portugol.jar → Editor de Texto → correr o exemplo com F3.

guia-instalar-portugol.pdf

VERIFICAR O JAVA NO TERMINAL

```
java -version
```

Não conseguiram?

Avisem no chat do Teams (ou por WhatsApp). Resolvemos nos primeiros minutos da sessão 2.

FLAG

RESUMO

O que levamos de hoje

1

Programar ≠ escrever código

É resolver problemas com passos precisos. A linguagem é só a ferramenta.

2

A ordem faz parte da solução

Sequência, decisão e repetição chegam para qualquer algoritmo.

3

Algoritmos têm regras

Finitude, definitude, entrada, saída e eficácia.

4

Primeiro no papel

Objetivo → decompor → testar → só depois codificar.

FLAG

www.flag.pt

LISBOA Edifício Mirage, R. Dr. Eduardo Neves, Nº3, 1050-077 Lisboa email: querosabermais@flag.pt Tel. 213 560 606

PORTO Rua Oliveira Monteiro, Nº 168, 4050–438 Porto email: querosabermais@porto.flag.pt Tel. 226 094 789

uma marca

Rumos training | Knowledge sharing